

04_variables-de-entorno

[#linux](#) [#bash](#) [#python](#) [#fastapi](#) [#docker](#) [#PostgreSQL](#) [#SQLAlchemy](#) [#alembic](#) [#backend](#) [#SIEM](#)

1 Objetivo de la nota

Esta nota analiza las variables de entorno utilizadas en el laboratorio SIEM MVP.

El objetivo es entender qué valores necesita el proyecto para conectarse a PostgreSQL, ejecutar el backend, exponer los puertos de la API y Adminer, y mostrar información de versión o build desde el endpoint `/info`.

Las variables de entorno permiten separar la configuración del código fuente.

Esto es importante porque evita escribir valores sensibles directamente dentro del código Python, Dockerfile, Alembic o archivos de la aplicación.

2 Archivos relacionados

Los archivos relacionados con variables de entorno son:

```
.env
.env.example
docker/.env
backend/alembic.ini
backend/app/db/database.py
backend/app/api/routes/info.py
docker/compose.yml
```

La relación general es:

```
.env / docker/.env / .env.example
    ↓
variables de entorno
    ↓
Docker / FastAPI / SQLAlchemy / Alembic
    ↓
ejecución del laboratorio
```

3 Archivo `.env.example`

El archivo mostrado contiene:

```
POSTGRES_DB=siem
POSTGRES_USER=siem
POSTGRES_PASSWORD=change_me
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem

API_PORT=8000
ADMINER_PORT=8080

APP_VERSION=0.1.0
GIT_SHA=unknown
BUILD_TIME=unknown
```

Este archivo funciona como plantilla.

No contiene credenciales reales de producción, sino valores de ejemplo.

Su función es indicar qué variables necesita el proyecto para ejecutarse.

4 Diferencia entre `.env`, `.env.example` y `docker/.env`

En el proyecto existen varios archivos relacionados:

```
.env
.env.example
docker/.env
```

La diferencia es importante.

.env.example

Es una plantilla.

Debe incluirse en la entrega y en el repositorio.

Sirve para que otra persona pueda crear su propio `.env`.

Ejemplo:

```
cp .env.example .env
```

.env

Es un archivo real de entorno.

Puede contener valores locales, contraseñas o configuración específica de la máquina.

No debería incluirse en la entrega final ni subirse al repositorio.

docker/.env

Es un archivo real de entorno usado por Docker Compose.

Puede contener valores reales para levantar los servicios.

Tampoco debería incluirse en la entrega si contiene configuración local.

5 Variable POSTGRES_DB

```
POSTGRES_DB=siem
```

Define el nombre de la base de datos PostgreSQL.

En este caso, la base de datos se llama:

```
siem
```

Esta variable se usa para inicializar PostgreSQL dentro del entorno Docker.

Relación:

```
POSTGRES_DB
  ↓
PostgreSQL crea base de datos
  ↓
backend se conecta a esa base
```

6 Variable POSTGRES_USER

```
POSTGRES_USER=siem
```

Define el usuario de PostgreSQL.

En este caso:

```
siem
```

Este usuario será utilizado por el backend para conectarse a la base de datos.

Relación:

```
POSTGRES_USER
  ↓
usuario PostgreSQL
  ↓
DATABASE_URL
  ↓
SQLAlchemy
```

7 Variable POSTGRES_PASSWORD

```
POSTGRES_PASSWORD=change_me
```

Define la contraseña del usuario PostgreSQL.

En `.env.example` aparece como:

```
change_me
```

Esto indica que es un valor de ejemplo y que debería cambiarse en un entorno real.

No conviene dejar contraseñas reales en archivos entregables.

8 Variable DATABASE_URL

```
DATABASE_URL=postgresql+psycopg://siem:change_me@db:5432/siem
```

Esta es una de las variables más importantes del backend.

Define la cadena de conexión completa a PostgreSQL.

Se utiliza por:

```
SQLAlchemy
Alembic
backend FastAPI
```

9 Desglose de DATABASE_URL

La cadena:

```
postgresql+psycopg://siem:change_me@db:5432/siem
```

puede descomponerse así:

```
postgresql+psycopg
  ↓
tipo de base de datos y driver

siem
  ↓
usuario
```

```
change_me
  ↓
contraseña

db
  ↓
host del servicio PostgreSQL dentro de Docker

5432
  ↓
puerto interno de PostgreSQL

siem
  ↓
nombre de la base de datos
```

Visualmente:

```
postgresql+psycopg://usuario:contraseña@host:puerto/base_datos
```

En este proyecto:

```
usuario      → siem
contraseña   → change_me
host         → db
puerto      → 5432
base_datos   → siem
```

10 Importancia del host db

Dentro de Docker Compose, los servicios se comunican usando el nombre del servicio como hostname.

Por eso la URL usa:

```
db
```

en lugar de:

```
localhost
```

Dentro del contenedor del backend:

```
db
  ↓
servicio PostgreSQL
```

Si el backend se ejecutara fuera de Docker, posiblemente habría que usar:

```
localhost
```

o la IP/host donde esté PostgreSQL.

11 Uso de DATABASE_URL en SQLAlchemy

En el archivo:

```
backend/app/db/database.py
```

se usa la variable `DATABASE_URL`.

El código relevante es:

```
DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL no está definido")

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

Esto significa que el backend no tiene la conexión hardcodeada.

La lee desde el entorno.

Flujo:

```
DATABASE_URL
  ↓
os.getenv("DATABASE_URL")
  ↓
create_engine()
  ↓
SessionLocal
  ↓
get_db()
  ↓
endpoints FastAPI
```

1 2 Control si falta DATABASE_URL

El código incluye:

```
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL no está definido")
```

Esto es importante.

Si la variable no existe, la aplicación falla de forma explícita.

Es mejor que fallar silenciosamente o producir errores más difíciles de interpretar después.

El mensaje indica claramente el problema:

```
DATABASE_URL no está definido
```

1 3 Uso de DATABASE_URL en Alembic

El archivo:

```
backend/alembic.ini
```

incluye:

```
sqlalchemy.url = %(DATABASE_URL)s
```

Y también comenta:

```
# No hardcodeamos credenciales aquí.
# Alembic leerá la URL desde la variable de entorno DATABASE_URL.
```

Esto significa que Alembic también usa la misma variable para conectarse a PostgreSQL.

Relación:

```
DATABASE_URL
  ↓
alembic.ini
  ↓
Alembic
  ↓
migraciones
  ↓
PostgreSQL
```

La ventaja es que la API y las migraciones usan la misma configuración de conexión.

1 4 Variable API_PORT

```
API_PORT=8000
```

Define el puerto en el que se expone la API desde Docker hacia la máquina local.

El backend internamente se ejecuta en:

```
8000
```

Con Docker Compose, esta variable puede usarse para publicar el puerto.

Resultado esperado:

```
http://localhost:8000
```

Desde ahí se accede a:

```
http://localhost:8000/docs
http://localhost:8000/health
http://localhost:8000/alerts
```

1 5 Variable ADMINER_PORT

```
ADMINER_PORT=8080
```

Define el puerto de Adminer.

Adminer es una herramienta web para consultar la base de datos PostgreSQL.

Si Docker Compose lo expone en ese puerto, se accedería desde:

```
http://localhost:8080
```

Relación:

```
ADMINER_PORT
  ↓
Docker Compose
  ↓
Adminer
  ↓
gestión visual de PostgreSQL
```

1 6 Variable APP_VERSION

```
APP_VERSION=0.1.0
```

Define la versión de la aplicación.

Esta variable se usa en el endpoint:

```
GET /info
```

En el archivo:

```
backend/app/api/routes/info.py
```

aparece:

```
"version": os.getenv("APP_VERSION", "0.1.0")
```

Esto significa que, si la variable existe, se usa su valor.

Si no existe, se devuelve por defecto:

```
0.1.0
```

17 Variable GIT_SHA

```
GIT_SHA=unknown
```

Define la referencia del commit Git asociado a la versión desplegada.

En el endpoint `/info` se usa así:

```
"git_sha": os.getenv("GIT_SHA", "unknown")
```

En este MVP aparece como:

```
unknown
```

Esto es aceptable en un entorno académico.

En un despliegue más avanzado, podría rellenarse automáticamente con el hash del commit.

18 Variable BUILD_TIME

```
BUILD_TIME=unknown
```

Define el momento de construcción o despliegue.

En `/info` se usa así:

```
"build_time": os.getenv("BUILD_TIME", "unknown")
```

En este proyecto aparece como:

```
unknown
```

En una evolución futura, podría establecerse automáticamente durante el build Docker.

19 Relación con el endpoint `/info`

El archivo:

```
backend/app/api/routes/info.py
```

devuelve:

```
return {
  "name": "siem-backend",
  "version": os.getenv("APP_VERSION", "0.1.0"),
  "git_sha": os.getenv("GIT_SHA", "unknown"),
  "build_time": os.getenv("BUILD_TIME", "unknown"),
  "utc_now": datetime.now(timezone.utc).isoformat(),
}
```

Por tanto, las variables:

```
APP_VERSION
GIT_SHA
BUILD_TIME
```

sirven para informar sobre la versión ejecutada.

Ejemplo conceptual de respuesta:

```
{
  "name": "siem-backend",
  "version": "0.1.0",
  "git_sha": "unknown",
  "build_time": "unknown",
  "utc_now": "2026-05-26T12:00:00+00:00"
}
```

2.0 Relación con el endpoint `/health`

El endpoint:

```
GET /health
```

comprueba la conexión con la base de datos.

Aunque no usa directamente todas las variables, depende de que `DATABASE_URL` esté correctamente configurado.

Flujo:

```
DATABASE_URL
  ↓
engine SQLAlchemy
  ↓
SessionLocal
  ↓
get_db()
  ↓
/health ejecuta SELECT 1
  ↓
respuesta ok
```

Si `DATABASE_URL` está mal configurado, `/health` fallará.

2.1 Relación entre variables y Docker

Las variables de entorno permiten a Docker Compose levantar los servicios con configuración externa.

Relación conceptual:

```
.env / docker/.env
  ↓
```



```
docker compose
  ↓
servicio db
  ↓
servicio api
  ↓
servicio adminer
```

Variables como:

```
POSTGRES_DB
POSTGRES_USER
POSTGRES_PASSWORD
API_PORT
ADMINER_PORT
DATABASE_URL
```

son necesarias para que los contenedores se comuniquen correctamente.

2.2 Relación entre variables y backend

El backend depende especialmente de:

```
DATABASE_URL
APP_VERSION
GIT_SHA
BUILD_TIME
```

Uso:

```
DATABASE_URL
  ↓
conexión a PostgreSQL

APP_VERSION / GIT_SHA / BUILD_TIME
  ↓
endpoint /info
```

La variable crítica es:

```
DATABASE_URL
```

Sin ella, el backend no puede crear el engine de SQLAlchemy.

2.3 Buenas prácticas aplicadas

El proyecto aplica varias buenas prácticas:

- No hardcodear credenciales dentro de alembic.ini.
- Usar DATABASE_URL como variable de entorno.
- Incluir .env.example como plantilla.
- Excluir .env real de la entrega.
- Separar configuración de código fuente.
- Usar valores por defecto seguros para /info.

Esto permite que el proyecto sea más portable y más limpio.

2.4 Archivos que deben entregarse

Debe entregarse:

```
.env.example
```

Porque sirve como plantilla.

También deben entregarse los archivos que consumen esas variables:

```
backend/app/db/database.py
backend/app/api/routes/info.py
backend/alembic.ini
docker/compose.yml
```

2.5 Archivos que no deben entregarse

No deberían entregarse:

```
.env
docker/.env
```

Motivo:

- Pueden contener credenciales reales.
- Son específicos del entorno local.
- No son necesarios si existe `.env.example`.

Para la entrega, lo correcto es incluir la plantilla y documentar cómo crear el archivo real.

2.6 Flujo para recrear variables desde plantilla

Una persona que reciba el proyecto podría hacer:

```
cp .env.example .env
```

Después podría revisar y modificar:

```
POSTGRES_PASSWORD=change_me
API_PORT=8000
ADMINER_PORT=8080
```

En Docker, si el Compose espera un `.env` dentro de `docker/`, podría hacerse también:

```
cp .env.example docker/.env
```

o ajustar la ruta según lo indique el README.

2.7 Comandos útiles

Ver plantilla de variables:

```
cat .env.example
```

Comprobar si existe `.env`:

```
ls -la .env
```

Comprobar si existe `docker/.env`:

```
ls -la docker/.env
```

Ver variable `DATABASE_URL` en la shell actual:

```
echo $DATABASE_URL
```

Probar endpoint `/info`:

```
curl http://localhost:8000/info
```

Probar endpoint `/health`:

```
curl http://localhost:8000/health
```

Buscar uso de variables de entorno en el backend:

```
grep -R "os.getenv" -n backend/app
```

Buscar `DATABASE_URL` en el proyecto:

```
grep -R "DATABASE_URL" -n .
```

28 Resumen técnico

Las variables de entorno permiten configurar el laboratorio sin modificar el código fuente.

El archivo `.env.example` documenta las variables necesarias:

```
POSTGRES_DB
POSTGRES_USER
POSTGRES_PASSWORD
DATABASE_URL
API_PORT
ADMINER_PORT
APP_VERSION
GIT_SHA
BUILD_TIME
```

La variable más importante es:

```
DATABASE_URL
```

porque conecta SQLAlchemy y Alembic con PostgreSQL.

Las variables `APP_VERSION`, `GIT_SHA` y `BUILD_TIME` alimentan el endpoint `/info`.

Los archivos `.env` y `docker/.env` son configuración real local y no deberían entregarse. En cambio, `.env.example` sí debe formar parte del proyecto porque permite reconstruir la configuración de forma segura.